In task 3 we implemented a partial evaluation that optimized both operrands of each binary operation, and if both are constants it evaluates the operation and replaces it with a single constants… Therefore nested expressions like 4+3*2 for instance becomes 10 right away making it effeciently optimized.
To prevent it from falling back to Python's default operator's we used the interpreter's (to_int64, trunc_div, trunc_mod) to make sure the result follows a 64-bit wrap and trunct so it truncates towards zero instead of negative infinity.

We unit tested our implementation using test_partial_eval.py which checks that optimized LangVar programs still meet their expectations including errors, we used claude to write ops_script.py a small program that goes through all the programs in progs/ including five other additional tests from us (task3_1→ 5.odc) and spit out how many operations each program had before and after optimization. Here is a small sample of optimized tests and how much (percent wise) were all the programs optimized.

Program	Before	After	Reduction
`int/20_all_operators`	9	0	100%
`var/13_all_arithmetic_operators`	7	4	42.9%
`var/task3_1`	5	0	100%
`var/task3_2`	1	1	0%
`var/task3_3`	3	0	100%
`var/task3_4`	5	0	100%
`var/task3_5` (`1/0`)	1	1	0%
All programs	**227**	**141**	**37.9%**